PRACTICALLY EFFICIENT LINEAR-TIME PROTOCOLS FOR SERVER-AIDED PRIVATE SET UNION AND THIRD PARTY PRIVATE SET OPERATIONS

FOO YEE YEO AND JASON H. M. YING

Abstract. We present protocols for server-aided private set union (PSU), third-party private set difference (TP-PSD) and third-party private symmetric difference (TP-PSymD). In a third-party setting, the receiver who obtains the output is an external inputless party with two other participating input parties. The protocols for third-party private set operations presented in this work are significantly more efficient than that of Yeo and Ying (USENIX '25). Our results improve upon the above work in both the computational complexity and practical performances. Moreover, our protocols demonstrate practical gains by achieving substantially quicker running times as well as the ability to run on much larger sets. Our server-aided private set union protocol is several times faster than existing state-of-the-art two-party private set union protocols.

1. Introduction

1.1. **Background.** Performing private operations over sets is essential in numerous diverse applications. Several useful set operations include intersection, union, set difference and symmetric difference. Each of these functionalities requires a distinct protocol to achieve the desired privacy requirements. Private set operations arise in a variety of settings. There is the conventional two party setting such that one or both parties obtain the output as well as a generalization of that to multiple parties in the multi-party setting. There is also a server assisted setting whereby an external server assists the two participating parties in obtaining the required output without disclosing any information to the assisting server. Finally, there is the third-party setting in which an external inputless party obtains the output without disclosing any information to the participating parties.

1.2. **Private set union.** The initial design of a private set union (PSU) protocol was due to Kisser and Song [24] and has since spanned a series of notable works [12, 17, 23, 26, 49]. The PSU protocol of Kissner and Song [24] represents each party's set elements as roots of a polynomial and is based on additively homomorphic encryption. The resulting complexities are both quadratic in communication and computation. A subsequent protocol due to Frikken [17] is also based on polynomial representation but which achieves linear communication complexity. Davidson and Cid [12] designed a protocol utilizing an encrypted Bloom filter and additively homomorphic encryption. Their protocol attains linear communication complexity and a linear computational cost consisting of public key operations. Despite achieving linear complexities, the protocol incurs a large run time due to the usage of expensive public key operations. Several subsequent works [23, 26, 49] aim at improving the concrete running time of private set union protocols.

We introduce a server-aided private set union protocol whereby an external server assists the two participating parties in finding the union of elements of their private

datasets without revealing any other information to all parties. Having a server to aid the protocol execution significantly reduces the total run time especially on large datasets. This is crucial for numerous time-sensitive applications such as in cyber risk assessment [21, 27]. In such scenarios, each organization maintains a list of blacklisted or suspicious IP addresses, and both seek to obtain a joint list for better threat mitigations. A server-aided private set union protocol is particularly suitable to be deployed in this scenario to handle large datasets while enabling both parties to obtain the outcome efficiently in a privacy-preserving manner.

1.3. **Third-party private set operations.** Private set intersection (PSI) is a well-established secure two-party computation cryptographic technique that enable parties to obtain the intersection outcome of their inputs without revealing other information [2, 7, 8, 10, 13, 16, 19, 24, 25, 29, 35–41, 43, 50, 51]. More recently, a variant of private set intersection known as third-party private set intersection (TP-PSI) is introduced [46], which incorporates a twist to the conventional two-party PSI model. In that setting, there are a total of three parties: one party is an external third party Q who performs the role of the receiver while the remaining two parties P_1, P_2 are senders with input sets S_1 and S_2 respectively. The functionality of the TP-PSI results in the external party obtaining the outcome of the intersection set $S_1 \cap S_2$ held by the two senders without revealing other information.

There have since been several progress in the domain of TP-PSI, including attaining a round-optimal protocol [48], achieving linear complexity [9], and improved efficiency in the unbalanced setting where the set sizes of the senders differ greatly [28]. In a recent work [47], the notion of TP-PSI is extended to the multi-party setting, and new third party private set operations are introduced, namely private set difference (TP-PSD) and private symmetric difference (TP-PSymD), whereby the receiver obtains $S_1 \setminus S_2$ and $S_1 \triangle S_2$, respectively. However, their proposed TP-PSD and TP-PSymD protocols incur at least quadratic complexities, and the efficiencies do not scale well with large set sizes.

There are several real-world applications for the third-party private set difference and private symmetric difference functionalities. In the event of a pandemic, public health authorities may issue contact tracing tokens which record proximity location data between individuals for contract tracing purposes. The public health authority is then able to notify close contacts of known infected individuals. Since there is typically a time gap from the onset of infection to eventual diagnosis, many individuals might have already been in close contact and some might have undergone testing during that time frame. As for those close contacts that have not undergone testing, they should be notified of potential exposure by the health authority. The third-party private set difference protocol can thus be set up whereby the external party Q is the public health authority, P_1 is an individual with input S_1 consisting of a list of close contacts in the time frame of interest, while S_2 correspond to the testing centers with input S_2 consisting of a database of people who have undergone testing in that same time frame. In this way, the public health authority is able to notify the list of people who have been in close contact with an infected but have yet to undergo testing. The advantages in this TP-PSD formulation are that no additional unnecessary information is exposed to the public health authority and reduces the logistics by notifying a smaller subset of individuals who have yet to undergo testing as compared to the entire list of close contacts.

TP-PSymD has applications in medical research for studying the effectiveness of a single round treatment versus multiple rounds. The medical research institution carrying out this study takes up the role of the external party Q, while the specialized medical facilities which can administer the treatments assume the roles P_1 and P_2. The sets S_1 and S_2 consist of the list of patients who have undergone exactly one round of treatment in P_1 and P_2 respectively. In this scenario, the TP-PSymD protocol enables the research institution to obtain the list of patients who have undergone exactly one round of treatment in either of these facilities without exposing other information.

1.4. **Our contributions.** We present a total of 4 protocols that achieve various privacy-preserving set operations in this work: 1 for server-aided private set union, 2 for third-party private set difference and 1 for third-party private symmetric difference.

We benchmark our server-aided private set union protocol, and the results obtained demonstrate significant efficiency improvements over the current most efficient state-of-the art private set union protocols [23, 49]. To the best of knowledge, our server-aided private set union protocol is the first of its kind. Our protocol achieves a speedup of approximately 2.5×–3.0× over current state-of-the-art two-party private set union protocols [23, 49] in single-threaded mode, and 4.5×–5.1× speedup in multithreaded mode with 8 threads.

We present 2 private set difference protocols. The first protocol can be instantiated in two ways: one optimized for overall performance, and another which has low communication and computational costs for Q. The second protocol is designed to have low communication and computational costs for P_1. Similarly, our symmetric difference protocol can be instantiated in different ways to either obtain a protocol which is fastest for general use, or a protocol which has low communication and computational costs for Q. The protocols which have low communication and computational costs for a particular party are designed so that they can be deployed on edge devices with limited compute resources.

Our protocols vastly improve upon the state-of-the-art third-party private set difference and private symmetric difference protocols by using new techniques. In particular, our protocols have linear complexities, as compared to the protocols by Yeo and Ying [47] which have complexity $O(n^{\omega+o(1)})$, where n is the size of each dataset and ω is the exponent of matrix multiplication.[1] Furthermore, our protocols also have much better practical performance compared to previous protocols. Our protocols are the first protocols to be practical for set sizes of up to $n = 2^{24}$.

In addition, all our protocols benefit from multithreading to a significantly greater extent compared to existing works in private set union [23, 49], third-party private set difference [47] and third-party private symmetric difference [47]. This results in larger speedups on systems with greater computing resources. Our protocols also utilize lightweight cryptographic primitives, using at most a small number of expensive public key operations that is independent of the input size. Finally, with minor changes, each protocol type presented in this work can be adapted to achieve information-theoretic security.

1.5. **Organization.** In Section 2, we provide formal definitions of the ideal functionalities for the various types of protocols we consider in this work, and introduce some of the key cryptographic primitives employed in our constructions. We then present our main server-aided PSU, TP-PSD and TP-PSymD protocols in Sections

[1]It is shown by Alman et al. [1] that $2 \leq \omega < 2.371339$.

3, 4 and 6, respectively. Section 5 describes a variant of our TP-PSD protocol that achieves greater efficiency when P_1 is resource-constrained. Finally, in Section 7, we present experimental results for our protocols under various settings and compare their performance with prior work.

2. Preliminaries

2.1. **Definitions.** We begin by defining the three types of protocols considered in this paper. Each protocol involves three parties: P_1 with an input set $S_1 \subseteq \{0,1\}^*$, P_2 with an input set $S_2 \subseteq \{0,1\}^*$, and Q with no input. The three types of protocols are:

- server-aided private set union (PSU), in which P_2 outputs the set union $S_1 \cup S_2$,
- third-party private set difference (TP-PSD), in which Q outputs the set difference $S_1 \setminus S_2$,
- third-party private symmetric difference (TP-PSymD), in which Q outputs the symmetric difference $S_1 \triangle S_2$.

We employ the universal composability (UC) framework, originally introduced by Canetti [4], to formalize the ideal functionalities underlying our protocols and to define their security properties. A simpler and more restricted variant of this framework was later proposed by Canetti et al. [5].

The ideal functionalities $\mathcal{F}_{\mathsf{PSU}}$, $\mathcal{F}_{\mathsf{TP\text{-}PSD}}$ and $\mathcal{F}_{\mathsf{TP\text{-}PSymD}}$ for server-aided PSU, TP-PSD and TP-PSymD are described in Figures 1, 2 and 3 respectively.

Parameters: Parties P_1, P_2 and Q. Size n of each dataset.
Functionality:

(1) Upon receiving input $S_1 = \{x_1, \ldots, x_n\} \subseteq \{0,1\}^*$ from P_1, record S_1.
(2) Upon receiving input $S_2 = \{y_1, \ldots, y_n\} \subseteq \{0,1\}^*$ from P_2, record S_2.
(3) Send $S_1 \cup S_2$ to P_2.

Figure 1. $\mathcal{F}_{\mathsf{PSU}}$ ideal functionality

Parameters: Parties P_1, P_2 and Q. Size n of each dataset.
Functionality:

(1) Upon receiving input $S_1 = \{x_1, \ldots, x_n\} \subseteq \{0,1\}^*$ from P_1, record S_1.
(2) Upon receiving input $S_2 = \{y_1, \ldots, y_n\} \subseteq \{0,1\}^*$ from P_2, record S_2.
(3) Send $S_1 \setminus S_2$ to Q.

Figure 2. $\mathcal{F}_{\mathsf{TP\text{-}PSD}}$ ideal functionality

Parameters: Parties P_1, P_2 and Q. Size n of each dataset.
Functionality:

(1) Upon receiving input $S_1 = \{x_1, \ldots, x_n\} \subseteq \{0,1\}^*$ from P_1, record S_1.
(2) Upon receiving input $S_2 = \{y_1, \ldots, y_n\} \subseteq \{0,1\}^*$ from P_2, record S_2.
(3) Send $S_1 \triangle S_2$ to Q.

Figure 3. $\mathcal{F}_{\mathsf{TP\text{-}PSymD}}$ ideal functionality

2.2. **Additive secret sharing.** Secret sharing is a method of distributing shares of a secret to a group of individuals such that each party obtains no information of the secret, and whereby the secret can be reconstructed when a designated number of parties pool their shares together. The first such secret sharing scheme is proposed by Shamir [44]. In the case where all parties' shares are required to reconstruct the secret, a more efficient additive secret sharing scheme can be applied. Shamir's secret sharing scheme and additive secret sharing are information theoretically secure.

Secret sharing is a core component in secure multi-party computation (MPC) protocols. A basic operation on shares is multiplying secret shared data. In other words, given shares of x and y, we want to compute shares of xy without reconstructing (or leaking any information about) either x or y. We describe the ideal functionality of such an operation in Figure 4.

Parameters: A finite field $\mathbb{F}$.
Functionality:

(1) Upon receiving inputs $\alpha, \gamma \in \mathbb{F}$ from P_1, record α, γ.
(2) Upon receiving inputs $\beta, \delta \in \mathbb{F}$ from P_2, record β, δ.
(3) Pick $\varepsilon, \zeta \leftarrow \mathbb{F}$ uniformly at random such that $\varepsilon + \zeta = (\alpha + \beta)(\gamma + \delta)$.
(4) Send ε to P_1 and ζ to P_2.

FIGURE 4. $\mathcal{F}_{\mathsf{ssMult}}$ ideal functionality

In practice, the functionality $\mathcal{F}_{\mathsf{ssMult}}$ can be achieved via the use of Beaver triples [3]. Beaver triples can either be generated by a trusted third-party, or in the absence of a trusted third-party, using techniques such as oblivious linear evaluation [20] or additive homomorphic encryption [11].

2.3. **Oblivious polynomial evaluation.** Oblivious polynomial evaluation (OPE) is a useful primitive introduced by Naor and Pinkas [31, 32]. OPE involves two parties: a sender S whose input is a polynomial over some field and a receiver R whose input is an element x over the same field. An OPE protocol enables R to obtain the evaluation of the polynomial at x, without S learning the private input x or R learning the polynomial.

We shall use a variant of OPE, in which, the parties instead obtain shares of the evaluation result. We shall call this a oblivious polynomial evaluation with secret shared output, or ssOPE, for short. The ideal functionality $\mathcal{F}_{\mathsf{ssOPE}}$ is described in Figure 5.

Parameters: A finite field $\mathbb{F}$ and a non-negative integer d.
Functionality:

(1) Upon receiving input $p(X) \in \mathbb{F}[X]$ of degree $\leq d$ from P_1, record $p(X)$.
(2) Upon receiving input $x \in \mathbb{F}$ from P_2, record x.
(3) Pick $\alpha, \beta \leftarrow \mathbb{F}$ uniformly at random such that $\alpha + \beta = p(x)$.
(4) Send α to P_1 and β to P_2.

FIGURE 5. $\mathcal{F}_{\mathsf{ssOPE}}$ ideal functionality

We can modify certain existing OPE protocols to yield ssOPE protocols. For example, Chang and Lu [6] introduced two OPE protocols and both of them can be easily adapted to provide the functionality $\mathcal{F}_{\mathsf{ssOPE}}$.

In the same paper, Chang and Lu also introduced the idea of an OPE protocol that enlists the help of a third party so as to avoid the use of oblivious transfers, a cryptographic primitive used in many OPE protocols. We adapt the protocol by Chang and Lu to design a protocol that achieves the ssOPE functionality with the help of a third party H. We shall refer to these as assisted ssOPE protocols.

Our protocol works by having P_1 send masked polynomial coefficients and P_2 send masked powers of the input x to H. The masking of the constant term is carefully designed to ensure the correctness of the protocol. Using these masked inputs, H can compute a masked version of the result when the polynomial is evaluated at x, and generate shares of this masked result to send to P_1 and P_2. With knowledge of the random values used to mask the inputs, P_1 and P_2 can then create shares of the actual evaluation result. Finally, the shares are rerandomized by P_1 and P_2 so that they are independent of the messages received by H. This rerandomization is necessary to make the protocol secure against a corrupted H.

Let $\mathbb{F}$ be a finite field, and suppose P_1 has a polynomial $p(X) = a_d X^d + \ldots + a_0 \in \mathbb{F}[X]$ of degree at most d, and P_2 has an input $x \in \mathbb{F}$. We now present our assisted ssOPE protocol.

(1) P_2 picks random $r_1, \ldots, r_d, s_0, \ldots, s_d \xleftarrow{\$} \mathbb{F}$ and sends them to P_1.
(2) P_2 computes $x_i = x^i + r_i$ for $i \in [d]$ and sends $x_1, \ldots, x_d$ to H.
(3) P_1 computes $b_0 = a_0 + s_0 - \sum_{i\in[d]} a_i r_i$, $b_i = a_i + s_i$ for $i \in [d]$ and sends $b_0, \ldots, b_d$ to H.
(4) H picks a random $t \xleftarrow{\$} \mathbb{F}$, sends $y = b_0 + \sum_{i\in[d]} b_i x_i - t$ to P_1 and t to P_2.
(5) P_1 picks a random $u \xleftarrow{\$} \mathbb{F}$ and sends it to P_2.
(6) P_1 outputs $y + u$ and P_2 outputs $t - u - s_0 - \sum_{i\in[d]} x_i s_i$.

Protocol 1: An assisted ssOPE protocol

It is easy to check the correctness of the above protocol:

Proposition 1. *Protocol 1 is correct, i.e. P_1 and P_2 outputs shares of $p(x)$.*

Proof. We show this using a direct computation. The sum of the outputs of P_1 and P_2 is

$$\begin{aligned}
&(y+u) + (t - u - s_0 - \sum_{i\in[d]} x_i s_i) \\
&= (b_0 + \sum_{i\in[d]} b_i x_i - t + u) + (t - u - s_0 - \sum_{i\in[d]} x_i s_i) \\
&= b_0 + \sum_{i\in[d]} b_i x_i - s_0 - \sum_{i\in[d]} x_i s_i \\
&= (a_0 + s_0 - \sum_{i\in[d]} a_i r_i) + \sum_{i\in[d]} (a_i + s_i)(x^i + r_i) - s_0 - \sum_{i\in[d]} (x^i + r_i) s_i \\
&= a_0 + \sum_{i\in[d]} a_i x^i,
\end{aligned}$$

from which correctness follows. □

Proposition 2. *Protocol 1 is secure against a semi-honest adversary corrupting a single party.*

Proof. First, consider a corrupted P_1. The simulator $\mathcal{S}$ generates independent and uniformly random elements for the messages $r_1, \ldots, r_d, s_0, \ldots, s_d$ received by P_1 as these are jointly uniformly random in P_1's view in the real protocol. Let α be P_1's output from the ideal functionality. $\mathcal{S}$ picks a uniformly random element $u \xleftarrow{\$} \mathbb{F}$ as in the real protocol and computes y as $\alpha - u$. Since y is masked by t, it is independent of $r_1, \ldots, r_d, s_0, \ldots, s_d$. As a result, P_1's output in the real execution is also independent of these values. Hence, the messages generated by $\mathcal{S}$ are identically distributed to those in the real protocol.

Now, we construct a simulator $\mathcal{S}$ for a corrupted P_2. $\mathcal{S}$ picks independent and uniformly random elements for $r_1, \ldots, r_d, s_0, \ldots, s_d$, and computes $x_i = x^i + r_i$ for $i \in [d]$ as in the real protocol. Let β be P_2's output from the ideal functionality. The simulator $\mathcal{S}$ generates a uniformly random element for t and computes u as $u = t - s_0 - \sum_{i\in[d]} x_i s_i - \beta$. The messages generated by $\mathcal{S}$ are again identically distributed to those in the real protocol, since $r_1, \ldots, r_d, s_0, \ldots, s_d, t$, together with P_2's output, are independent and uniformly random in the real execution.

H receives $x_1, \ldots, x_d, b_0, \ldots, b_d$, which are uniformly random elements of $\mathbb{F}$ in H's view since x_i is masked by r_i and b_i is masked by s_i. Furthermore, the output is independent of the messages received by H since they are rerandomized using a uniformly random element u that is unknown to H. Hence, we can construct a simulator $\mathcal{S}$ for a corrupted H that simply simulates the messages received by H using independent and uniformly random elements of $\mathbb{F}$. □

We note that Protocol 1 is information-theoretically secure. As we shall see later, this allows us to design server-aided PSU, TP-PSD and TP-SymD protocols with information-theoretic security.

Since one approach to generate Beaver triples for realizing the $\mathcal{F}_{\mathsf{ssMult}}$ functionality is through oblivious linear evaluation, Protocol 1 immediately gives rise to an assisted ssMult protocol. This approach yields a significantly more efficient realization of the $\mathcal{F}_{\mathsf{ssMult}}$ functionality.

2.4. **Cuckoo hashing.** Cuckoo hashing is a hashing technique introduced by Pagh and Rodler [33,34] as an alternative to traditional collision resolution methods, that seeks to achieve expected constant-time lookups and high space efficiency. Their work presents several variations of the scheme. In this paper, we focus on the variant that utilizes a single hash table.

In this variant, two hash functions are used to map each element into one of two possible bins, such that each bin contains at most one element. This is achieved by the following process: whenever an element x' is designated in a bin which already contains another element x, the element x is evicted from its original bin location and reinserted into the bin determined by its alternate hash function. This eviction and reinsertion process continues until all elements are placed or a predefined threshold limit is exceeded. If the threshold is exceeded, the insertion procedure is aborted and considered a failure.

Subsequent research explored extensions of cuckoo hashing. Notably, a work by Fotakis et al. [14, 15] described and analyzed a generalization of cuckoo hashing which incorporates more than two hash functions to further reduce the probability of insertion failure. Cuckoo hashing has been utilized in multiple works [36, 38–40] to improve the practical efficiency of PSI protocols.

3. A Server-Aided Private Set Union Protocol

3.1. **An overview.** In this section, we describe a fast private set union (PSU) protocol for two input parties P_1 and P_2, aided by a third party Q. P_2 is the receiver in this protocol, i.e. P_2 obtains the set union $S_1 \cup S_2$. Since P_2 knows S_2, computing the set union $S_1 \cup S_2$ is equivalent to computing $S_1 \setminus S_2$.

The approach we use to determine the set difference $S_1 \setminus S_2$ is to apply an assisted oblivious polynomial evaluation protocol with secret shared output (i.e. an assisted ssOPE protocol — see Section 2.3), for which we have a very fast information theoretically secure instantiation, namely Protocol 1.

First, P_1 performs secret sharing on each element in S_1 and sends one of the two shares to P_2. Next, P_2 forms a polynomial $p(X)$ with roots at the elements of S_2, which they then evaluate at the elements of S_1 using the assisted ssOPE protocol. The result of an evaluation will be 0 if an element $s \in S_1$ lies also in S_2, and non-zero otherwise.

For each element $s \in S_1$, the shares α, β of s and the shares γ, δ of $p(s)$ are then "multiplied" to create shares ε, ζ of $sp(s)$. We again enlist Q's assistance to make this step more efficient.[2] Finally, P_1 sends the shares γ and ε to P_2, who can use them to recover s whenever $p(s) \neq 0$ (i.e. whenever $s \in S_1 \setminus S_2$) by computing

$$\frac{\varepsilon + \zeta}{\gamma + \delta} = \frac{sp(s)}{p(s)} = s.$$

The protocol, as described above, has quadratic complexity since we perform n evaluations using the assisted ssOPE protocol with a degree n polynomial. To reduce the complexity to linear and achieve a protocol with good practical performance, we incorporate hashing techniques commonly employed in PSI protocols.

The rough idea is to have P_1 hash his items into bins using cuckoo hashing [14,33] with 3 hash functions such that each bin contains at most a single element. P_2 uses the same hash functions to hash his items into bins using simple hashing. The above protocol is then performed multiple times, once on each bin. Since not all of P_1's bins are non-empty, we fill each empty bin with a dummy element so as not to leak information about S_1.

Since the result of cuckoo hashing depends on the entire set S_1 and not only on individual elements, it is necessary to hide from P_2 the information of which element ends up in which bin after P_1 applies cuckoo hashing to S_1. One way to do so is to apply an oblivious switching protocol [30], such as in a work by Garimella et al. [18]. However, we can again enlist the help of Q to improve efficiency. Instead of using oblivious switching, P_1 and Q uses a PRP to jointly shuffle the shares. This is achieved by having P_2 send his shares to Q, who shuffles it before sending it back to P_2.

To prevent P_2 from recovering the random shuffle that has been applied to the shares, it is necessary for P_1 and Q to rerandomize the shares. As we will see, the shares must still undergo a second randomization to prevent P_2 from inferring information about the random permutation using his knowledge of the polynomials used in the ssOPE protocol and the combined shares.

3.2. **Protocol details.** Suppose P_1 and P_2 have sets $S_1 \subseteq \{0,1\}^\ell$ and $S_2 \subseteq \{0,1\}^\ell$ respectively, each of size n. We embed $\{0,1\}^\ell$ into a finite field $\mathbb{F}$, and fix a dummy element $d \in \mathbb{F} \setminus \{0,1\}^\ell$.

[2]As noted in Section 2.3, an assisted ssOPE protocol naturally leads to an assisted ssMult protocol.

We choose some positive integer b such that hashing n items into b bins using cuckoo hashing with 3 hash functions succeeds except with negligible probability. We choose 3 independent random hash functions $h_1, h_2, h_3 : \mathbb{F} \to [b]$ to be used for both cuckoo hashing and simple hashing. Next, fix some positive integer m such that the probability of some bin containing more than m items is negligible when using 3 random hash functions to hash n items into B bins. Finally, let $\pi : \mathcal{K}_1 \times [b] \to [b]$ be a PRP.

Our server-aided PSU protocol is described in Protocol 2.

(1) P_1 uses cuckoo hashing with the hash functions h_1, h_2 and h_3 to hash the items of S_1 into b bins $B_1, \ldots, B_b$. Empty bins are filled with the dummy element d.
(2) P_2 uses simple hashing with the hash functions h_1, h_2 and h_3 to hash the items of S_2 into $B'_1, \ldots, B'_b$.
(3) P_1 generates additive shares α_i and β_i for the unique item x_i in bin B_i and sends β_i to P_2.
(4) For each $i \in [b]$, P_2 computes the polynomial
$$p_i(X) = (X - d) \prod_{y_i \in B'_i} (X - y_i).$$
(5) For each $i \in [b]$, P_1 and P_2 invoke $\mathcal{F}_{\mathsf{ssOPE}}$, assisted by Q, with
- P_1 having input x_i,
- P_2 having input $p_i(X)$.

Let γ_i and δ_i be the outputs obtained by P_1 and P_2 respectively from $\mathcal{F}_{\mathsf{ssOPE}}$.
(6) For each $i \in [b]$, P_1 and P_2 invoke $\mathcal{F}_{\mathsf{ssMult}}$, assisted by Q, with
- P_1 having inputs α_i and γ_i,
- P_2 having inputs β_i and δ_i,

to obtain shares ε_i and ζ_i for P_1 and P_2 respectively.
(7) P_2 sends δ_i and ζ_i to Q for $i \in [b]$.
(8) P_1 picks a random key $k \in \mathcal{K}_1$, random elements $r_i \xleftarrow{\$} \mathbb{F}\backslash\{0\}$, $s_i, t_i \xleftarrow{\$} \mathbb{F}$ and sends them to Q.
(9) P_1 and Q shuffle γ_i, ε_i and δ_i, ζ_i respectively using the permutation $\pi_k : [b] \to [b]$, i.e. they compute
$$\gamma'_i = \gamma_{\pi_k(i)}, \quad \varepsilon'_i = \varepsilon_{\pi_k(i)}, \quad \delta'_i = \delta_{\pi_k(i)}, \quad \zeta'_i = \zeta_{\pi_k(i)}.$$
(10) P_1 and Q rerandomizes the shares γ'_i, ε'_i, δ'_i and ζ'_i by computing
$$\gamma''_i = r_i(\gamma'_i + s_i), \qquad \varepsilon''_i = r_i(\varepsilon'_i + t_i) \qquad \text{(for } P_1\text{)},$$
and
$$\delta''_i = r_i(\delta'_i - s_i), \qquad \zeta''_i = r_i(\zeta'_i - t_i) \qquad \text{(for } Q\text{)}.$$
(11) P_1 sends γ''_i, ε''_i and Q sends δ''_i, ζ''_i to P_2 for $i \in [b]$.
(12) P_2 outputs
$$S_2 \cup \left\{ \frac{\varepsilon''_i + \zeta''_i}{\gamma''_i + \delta''_i} \text{ such that } \gamma''_i + \delta''_i \neq 0 : \ i \in [b] \right\}.$$

PROTOCOL 2: A fast linear-time server-aided PSU protocol

We note, from the above specification, that Protocol 2 has linear computational and communication costs. Furthermore, it is possible to modify Protocol 2 to

achieve information theoretic security by using information-theoretically secure protocols to realize $\mathcal{F}_{\mathsf{ssOPE}}$ and $\mathcal{F}_{\mathsf{ssMult}}$, and replacing the PRP with a truly random permutation. Information-theoretically secure protocols for $\mathcal{F}_{\mathsf{ssOPE}}$ and $\mathcal{F}_{\mathsf{ssMult}}$ can both be realized using Protocol 1.

3.3. **Choice of parameters.** The use of cuckoo hashing in PSI has been studied before by Pinkas et al. [40]. In that paper, the authors analyzed the failure probability of cuckoo hashing in various scenarios. In particular, for cuckoo hashing without a stash using 3 hash functions, the authors estimate that the choice of $b = 1.27n$ is sufficient to obtain a failure probability of 2^{-40}.

To choose an appropriate value for m, we want to ensure that, when hashing n items using 3 hash functions into b bins, the probability of having more than m items in some bin is negligible. This can be easily bounded as follows:

$$\Pr[\text{some bin has more than } m \text{ items}] \leq b\left(1 - \sum_{i=0}^{m} \binom{3n}{i} \left(\frac{1}{b}\right)^i \left(1 - \frac{1}{b}\right)^{3n-i}\right).$$

For set sizes n of up to 2^{26}, we find that choosing $m = 27$ provides a failure probability of $< 2^{-40}$. For smaller set sizes, it is possible to choose smaller values for m.

3.4. **Correctness and security.**

Proposition 3. *Protocol 2 is correct except with negligible probability.*

Proof. We start by making the trivial observation that $S_1 \cup S_2$ is the disjoint union of S_2 and $S_1 \setminus S_2$, so it suffices to show that the set

$$\left\{\frac{\varepsilon_i'' + \zeta_i''}{\gamma_i'' + \delta_i''} \text{ such that } \gamma_i'' + \delta_i'' \neq 0 : \ i \in [b]\right\}$$

obtained by P_2 in step 12 of the protocol equals $S_1 \setminus S_2$.

By the choices of b and m, steps 1 and 2 succeed except with negligible probability. We will consider two separate cases depending on whether a bin B_i contains an actual element x_i or the dummy element d.

First, we consider the case where a bin B_i has an actual item x_i. If $x_i \in S_2$, then x_i must be in the bin B_i', hence $(X - x_i)$ is a factor of the polynomial $p_i(X)$. This means that $p_i(x_i) = 0$, hence P_1 and P_2 obtain shares γ_i and δ_i such that $\gamma_i + \delta_i = p_i(x_i) = 0$. If we let $j = \pi_k^{-1}(i)$, then

$$\gamma_j'' + \delta_j'' = r_j(\gamma_j' + \delta_j') = r_j(\gamma_i + \delta_i) = 0,$$

in which case P_2 has no output for this particular value of j.

On the other hand, if $x_i \notin S_2$, then $(X - x_i)$ is not a factor of $p_i(X)$, hence $p_i(x_i) \neq 0$. As above, we still have $\gamma_i + \delta_i = p_i(x_i)$. Upon invoking $\mathcal{F}_{\mathsf{ssMult}}$ in step 6, P_1 and P_2 then obtain shares ε_i and ζ_i such that

$$\varepsilon_i + \zeta_i = (\alpha_i + \beta_i)(\gamma_i + \delta_i) = x_i p_i(x_i).$$

Thus, if we again let $j = \pi_k^{-1}(i)$, P_2 obtains the output

$$\frac{\varepsilon_j'' + \zeta_j''}{\gamma_j'' + \delta_j''} = \frac{r_j(\varepsilon_j' + \zeta_j')}{r_j(\gamma_j' + \delta_j')} = \frac{\varepsilon_i + \zeta_i}{\gamma_i + \delta_i} = \frac{x_i p_i(x_i)}{p_i(x_i)} = x_i.$$

Next, we consider the case where a bin B_i has a dummy element d. Note that, by construction, $(X - d)$ is always a factor of $p_i(X)$, hence $\gamma_i + \delta_i = p_i(d) = 0$. This means that, for $j = \pi_k^{-1}(i)$,

$$\gamma_j'' + \delta_j'' = r_j(\gamma_i + \delta_i) = 0,$$

so P_2 has no output for this value of j. □

Proposition 4. *Protocol 2 is secure in the* $(\mathcal{F}_{\mathsf{ssMult}}, \mathcal{F}_{\mathsf{ssOPE}})$*-hybrid model against a semi-honest adversary corrupting a single party.*

Proof. Consider the case where P_1 is corrupted. The simulator $\mathcal{S}$ samples α_i, β_i and k exactly as in the real protocol, and it simulates the messages γ_i and ε_i received by P_1 using independent and uniformly random elements of $\mathbb{F}$. It is clear that these simulated messages are identically distributed to those in the real execution, proving the security of Protocol 2 against a corrupted P_1.

Similarly, the protocol is secure against a corrupted Q since the only messages received by Q are δ_i, ζ_i, r_i, s_i, t_i and k. The simulator $\mathcal{S}$ simulates δ_i, ζ_i, s_i and t_i using independent and uniformly random elements of $\mathbb{F}$, simulates r_i using a uniformly random element of $\mathbb{F} \setminus \{0\}$, and simulates k using a randomly chosen key for the PRP π.

Let us now consider the case of a corrupted P_2. P_2 receives the messages β_i, δ_i, ζ_i, γ_i'', δ_i'', ε_i'' and ζ_i''. We note that β_i, δ_i and ζ_i are independent and uniformly random elements of $\mathbb{F}$, since they are shares of x_i, $p_i(x_i)$ and $x_i p_i(x_i)$ respectively. Fix some i, and let $j = \pi_k(i)$. Note that the pair (γ_i'', δ_i'') is uniformly random subject to the constraint that

$$\gamma_i'' + \delta_i'' = r_i(\gamma_j + \delta_j) = r_i p_j(x_j).$$

Since r_i is a uniformly random element of $\mathbb{F}\setminus\{0\}$ unknown to P_2, whenever $p_j(x_j) \neq 0$, the element $r_i p_j(x_j)$ is a uniformly random element of $\mathbb{F} \setminus \{0\}$ in P_2's view. Similarly, $(\varepsilon_i'', \zeta_i'')$ is uniformly random under the constraint

$$\varepsilon_i'' + \zeta_i'' = r_i(\varepsilon_j + \zeta_j) = r_i x_j p_j(x_j).$$

Furthermore, we note that γ_i'', δ_i'', ε_i'' and ζ_i'' are independent of β_i, δ_i and ζ_i, since they have been rerandomized using the random elements s_i and t_i, which are not known to P_2. Finally, we recall from the proof of Proposition 3 that $p_j(x_j) \neq 0$ if and only if $x_j \in S_1 \setminus S_2$.

We now explain how to construct a simulator $\mathcal{S}$ for a corrupted P_2. First, $\mathcal{S}$ picks independent and uniformly random elements $\beta_i, \delta_i, \zeta_i, \gamma_i'', \varepsilon_i'' \xleftarrow{\$} \mathbb{F}$ for each $i \in [b]$. Next, using P_2's input and output, $\mathcal{S}$ computes the elements $z_1, \ldots, z_d$ in P_2's output that do not lie in S_2. $\mathcal{S}$ then picks a random injective map $\rho : [d] \to [b]$, and simulates the messages δ_i'' and ζ_i'' received by P_2 as follows:

$$\begin{aligned} &\delta_i'' \xleftarrow{\$} \mathbb{F} \setminus \{-\gamma_i''\} && \text{if } i = \rho(j) \text{ for some } j \in [d], \\ &\delta_i'' = -\gamma_i'' && \text{otherwise,} \end{aligned}$$

and

$$\zeta_i'' = \begin{cases} z_j(\gamma_i'' + \delta_i'') - \varepsilon_i'' & \text{if } i = \rho(j) \text{ for some } j \in [d], \\ -\varepsilon_i'' & \text{otherwise.} \end{cases}$$

By the above argument, the messages generated by $\mathcal{S}$ in the ideal execution are computationally indistinguishable to those in the real execution, proving security against a corrupted P_2. □

4. A Linear Third Party Private Set Difference Protocol

4.1. **An overview.** Suppose P_1 and P_2 hold sets S_1 and S_2 respectively. The goal of a third-party private set difference (TP-PSD) protocol is for a third-party Q to learn the set difference $S_1 \setminus S_2$. Note that this problem is similar to that of server-aided PSU (see Section 3), which also involves 3 parties P_1, P_2 and Q. The

key distinction in their functionalities lies in the party who acts as the receiver. In TP-PSD, the third party Q receives the result $S_1 \setminus S_2$, whereas in server-aided PSU, the output is obtained by P_2.[3] Based on this observation, let us consider how we can modify Protocol 2, a server-aided PSU protocol, to achieve the TP-PSD functionality.

We begin in a similar manner as Protocol 2, where we have P_1 and P_2 hash their elements into bins using cuckoo hashing and simple hashing respectively, and P_1 filling each of his empty bins with a dummy element d. P_1 then generates shares α_i and β_i for the unique element x_i in each bin and sends one of the shares to P_2. For each of his bins, P_2 computes a polynomial $p_i(X)$ with roots at the elements in the bin. Following this, P_1 and P_2 invoke an ssOPE protocol to generate shares γ_i and δ_i of $p_i(x_i)$, which they then "multiply" by the shares of x_i to obtain shares ε_i and ζ_i of $x_i p_i(x_i)$.

If the shares γ_i, δ_i, ε_i and ζ_i are now sent to Q, he can use these shares to recover x_i whenever $p_i(x_i) \neq 0$, i.e. whenever $x_i \in S_1 \setminus S_2$, by computing

$$\frac{\varepsilon_i + \zeta_i}{\gamma_i + \delta_i} = \frac{x_i p(x_i)}{p(x_i)} = x_i.$$

However, to prevent Q from learning information about S_1 based on the result of cuckoo hashing performed by P_1, we need to conceal the mapping of elements to bins. Thus, instead, we have P_1 and P_2 shuffle their shares with a PRP using a common key before the shares are sent to Q. In this case, rerandomization of the shares is not required since Q never has access to the shares before this shuffling step, and thus cannot learn information about the random shuffle used even without rerandomizing the shares.

Nevertheless, there is still an undesired information leakage in the above protocol description. It is necessary to prevent Q from learning $p_i(x_i)$ since knowledge of both x_i and $p_i(x_i)$ leaks a small amount of information about the polynomial $p_i(X)$, and hence can conceivably leak information about S_2. To prevent this leakage, P_2 modifies the polynomial $p_i(X)$ used in the ssOPE protocol by multiplying it with a random non-zero element.

4.2. **Protocol details.** We use the same setup as Section 3.2. The detailed description of our first TP-PSD protocol is as follows:

(1) P_1 uses cuckoo hashing with the hash functions h_1, h_2 and h_3 to hash the items of S_1 into b bins $B_1, \ldots, B_b$. Empty bins are filled with the dummy element d.
(2) P_2 uses simple hashing with the hash functions h_1, h_2 and h_3 to hash the items of S_2 into $B'_1, \ldots, B'_b$.
(3) P_1 generates additive shares α_i and β_i for the unique item x_i in bin B_i and sends β_i to P_2.
(4) For each $i \in [b]$, P_2 picks a random $r_i \xleftarrow{\$} \mathbb{F} \setminus \{0\}$ and computes the polynomial
$$p_i(X) = r_i(X - d) \prod_{y_i \in B'_i} (X - y_i).$$
(5) For each $i \in [b]$, P_1 and P_2 invoke $\mathcal{F}_{\mathsf{ssOPE}}$ with
   - P_1 having input x_i,

[3]The output of P_2 in server-aided PSU is $S_1 \cup S_2$. However, computing $S_1 \cup S_2$ for P_2 is equivalent to computing $S_1 \setminus S_2$, since $S_1 \cup S_2$ is the disjoint union of S_2 and $S_1 \setminus S_2$.

- P_2 having input $p_i(X)$.

Let γ_i and δ_i be the outputs obtained by P_1 and P_2 respectively from $\mathcal{F}_{\mathsf{ssOPE}}$.

(6) For each $i \in [b]$, P_1 and P_2 invoke $\mathcal{F}_{\mathsf{ssMult}}$ with
- P_1 having inputs α_i and γ_i,
- P_2 having inputs β_i and δ_i,

to obtain shares ε_i and ζ_i for P_1 and P_2 respectively.

(7) P_1 picks a random key $k \in \mathcal{K}_1$ and sends k to P_2.

(8) P_1 and P_2 shuffle γ_i, ε_i and δ_i, ζ_i respectively using the permutation $\pi_k : [b] \to [b]$, i.e. they compute

$$\gamma'_i = \gamma_{\pi_k(i)}, \quad \varepsilon'_i = \varepsilon_{\pi_k(i)}, \quad \delta'_i = \delta_{\pi_k(i)}, \quad \zeta'_i = \zeta_{\pi_k(i)}$$

and send the shuffled elements to Q.

(9) Q outputs

$$\left\{ \frac{\varepsilon'_i + \zeta'_i}{\gamma'_i + \delta'_i} \text{ such that } \gamma'_i + \delta'_i \neq 0 : \ i \in [b] \right\}.$$

Protocol 3: A fast linear-time TP-PSD protocol

For the best efficiency, we can instantiate $\mathcal{F}_{\text{ssOPE}}$ and $\mathcal{F}_{\text{ssMult}}$ with protocols that are assisted by Q. Alternatively, they can instantiated with unasssisted protocols to obtain a TP-PSD protocol that is cheap for Q. In the latter case, it is clear that Q receives only $4b$ elements of $\mathbb{F}$, and the only computation Q performs is step 9 of the protocol.

4.3. **Correctness and security.**

Proposition 5. *Protocol 3 is correct except with negligible probability.*

Proof. By the choices of b and m, hashing succeeds except with negligible probability. Similar to the proof of Proposition 3, we observe that

$$\gamma_i + \delta_i = \begin{cases} p_i(x_i) & \text{if } B_i \text{ contains } x_i \neq d \text{ and } x_i \notin S_2, \\ 0 & \text{otherwise,} \end{cases}$$

and

$$\varepsilon_i + \zeta_i = \begin{cases} x_i p_i(x_i) & \text{if } B_i \text{ contains } x_i \neq d \text{ and } x_i \notin S_2, \\ 0 & \text{otherwise.} \end{cases}$$

Thus, if $x_i \in S_1 \setminus S_2$, then for $j = \pi_k^{-1}(i)$,

$$\frac{\varepsilon'_j + \zeta'_j}{\gamma'_j + \delta'_j} = \frac{\varepsilon_i + \zeta_i}{\gamma_i + \delta_i} = \frac{x_i p_i(x_i)}{p_i(x_i)} = x_i.$$

On the other hand, if B_i contains d, or if $x_i \in S_2$, then Q has no output for $j = \pi_k^{-1}(i)$ since $\gamma'_j + \delta'_j = \gamma_i + \delta_i = 0$. □

Proposition 6. *Protocol 3 is secure in the ($\mathcal{F}_{\mathsf{ssMult}}, \mathcal{F}_{\mathsf{ssOPE}}$)-hybrid model against a semi-honest adversary corrupting a single party.*

Proof. Protocol 3 is secure against a corrupted P_1 since the only messages received by P_1 are γ_i and ε_i, which are uniformly random elements of $\mathbb{F}$.

Similarly, the protocol is secure against a corrupted P_2 since the only messages received by P_2 are β_i, δ_i, ζ_i and k. The elements β_i, δ_i, ζ_i are uniformly random elements of $\mathbb{F}$, and k is a randomly chosen key for the PRP π.

Now, consider the case of a corrupted Q. Q receives the messages γ'_i, ε'_i from P_1 and δ'_i, ζ'_i from P_2. Fix some i, and let $j = \pi_k(i)$. The pair (γ'_i, δ'_i) is uniformly random subject to the constraint that

$$\gamma'_i + \delta'_i = \gamma_j + \delta_j = p_j(x_j).$$

Recall from the proof of Proposition 5 that $p_j(x_j) \neq 0$ if and only if $x_j \in S_1 \setminus S_2$. In this case,

$$p_j(x_j) = r_j(x_j - d) \prod_{y_j \in B'_j} (x_j - y_j)$$

is a uniformly random element of $\mathbb{F} \setminus \{0\}$ in Q's view since r_j is a uniformly random element of $\mathbb{F} \setminus \{0\}$ unknown to Q. Similarly, $(\varepsilon'_i, \zeta'_i)$ is uniformly random under the constraint

$$\varepsilon'_i + \zeta'_i = \varepsilon_j + \zeta_j = x_j p_j(x_j).$$

We now construct a simulator $\mathcal{S}$ for a corrupted Q. For each i, $\mathcal{S}$ picks independent and uniformly random elements $\gamma'_i, \varepsilon'_i \xleftarrow{\$} \mathbb{F}$. Let $\{z_1, \ldots, z_d\}$ be Q's output from the ideal functionality. $\mathcal{S}$ picks a random injective map $\rho : [d] \to [b]$, and computes

$$\begin{aligned} &\delta'_i \xleftarrow{\$} \mathbb{F} \setminus \{-\gamma'_i\} && \text{if } i = \rho(j) \text{ for some } j \in [d], \\ &\delta'_i = -\gamma'_i && \text{otherwise,} \end{aligned}$$

and

$$\zeta'_i = \begin{cases} z_j(\gamma'_i + \delta'_i) - \varepsilon'_i & \text{if } i = \rho(j) \text{ for some } j \in [d], \\ -\varepsilon'_i & \text{otherwise.} \end{cases}$$

□

5. A TP-PSD Protocol for Resource Constrained P_1

5.1. **An overview.** In the previous section, we have designed a TP-PSD protocol that can be instantiated in different ways to obtain either a protocol with the best overall performance, or a protocol more suitable for a resource constrained Q. In this section, we shall present a variant of Protocol 3 that has low computational and communication costs for P_1.

As the most costly portion of Protocol 3 is the ssOPE protocol, our aim is to design a protocol in which ssOPE can instead be performed entirely by P_2 and Q, without involving P_1. Recall that, in Protocol 3, the ssOPE protocol is invoked with input x_i, the unique item in bin B_i, and a polynomial $p_i(X)$ which is computed by P_2.

Since P_1's input set S_1 is private, we cannot simply have P_1 send x_i to Q. Instead, the idea is for P_1 and P_2 to first encrypt both x_i and $p_i(X)$ using a PRP (or a PRF) with a common key k. The encrypted x_i can then be sent to Q, who can engage in the ssOPE protocol with P_2 using the encrypted $p_i(X)$. Since both x_i and $p_i(X)$ are encrypted, the output of the ssOPE protocol has the same property as before, i.e. the shares output by the protocol sum to 0 if x_i is a root of $p_i(X)$ and a random value otherwise.

Note that P_1 still generates shares α_i and β_i of x_i, not shares of the encrypted version, as in Protocol 3. This does not leak information because these shares are never reconstructed.

Furthermore, recall that, in Protocol 3, the shares must be shuffled by P_1 and P_2 before they are sent to Q. Due to the properties of cuckoo hashing, this shuffling is necessary to prevent leakage of information about S_1. Even in this protocol variant,

shuffling must still be performed by P_1 and P_2. We move the shuffling step to earlier in the protocol before P_1 sends his encrypted items to Q.

5.2. **Protocol details.** We use a similar setup as the previous protocol. Furthermore, let $F : \mathcal{K}_2 \times \mathbb{F} \to \mathbb{F}$ be a PRP or a PRF. If F is a PRF, we will assume that distinct elements in the same bin map to distinct elements under F except with negligible probability. This can always be achieved by using a larger field $\mathbb{F}$ if necessary.

We present our TP-PSD protocol with low computational and communication costs for P_1 below:

(1) P_1 uses cuckoo hashing with the hash functions h_1, h_2 and h_3 to hash the items of S_1 into b bins $B_1, \ldots, B_b$. If B_i is empty, it is filled with the dummy element d.
(2) P_2 uses simple hashing with the hash functions h_1, h_2 and h_3 to hash the items of S_2 into $B'_1, \ldots, B'_b$.
(3) P_1 picks random keys $k_1 \in \mathcal{K}_1$, $k_2 \in \mathcal{K}_2$ and sends k_1, k_2 to P_2.
(4) P_1 generates additive shares α_i and β_i for the unique item x_i in B_i.
(5) P_1 shuffles α_i, β_i using the permutation $\pi_{k_1} : [b] \to [b]$, i.e. P_1 computes
$$\alpha'_i = \alpha_{\pi_{k_1}(i)}, \qquad \beta'_i = \beta_{\pi_{k_1}(i)},$$
and sends α'_i to Q, β'_i to P_2.
(6) P_1 computes $e_i = F_{k_2}(x_i)$, defines $e'_i = e_{\pi_{k_1}(i)}$ and sends e'_i to Q.
(7) For each $i \in [B]$, P_2 picks a random $r_i \xleftarrow{\$} \mathbb{F} \setminus \{0\}$ and computes the polynomial
$$p_i(X) = r_i(X - F_{k_2}(d)) \prod_{y_i \in B'_i} (X - F_{k_2}(y_i)).$$
(8) For each $i \in [B]$, P_2 and Q invoke $\mathcal{F}_{\mathsf{ssOPE}}$ with
- P_2 having input $p'_i(X) = p_{\pi_{k_1}(i)}(X)$, and
- Q having input e'_i.

Let γ'_i and δ'_i be the outputs obtained by Q and P_2 respectively from $\mathcal{F}_{\mathsf{ssOPE}}$.
(9) For each $i \in [B]$, P_2 and Q invoke $\mathcal{F}_{\mathsf{ssMult}}$ with
- P_2 having inputs β'_i and δ'_i, and
- Q having inputs α'_i and γ'_i.

to obtain shares ζ'_i and ε'_i respectively.
(10) P_2 sends δ'_i and ζ'_i to Q for $i \in [B]$.
(11) Q outputs
$$\left\{ \frac{\varepsilon'_i + \zeta'_i}{\gamma'_i + \delta'_i} \text{ such that } \gamma'_i + \delta'_i \neq 0 : \ i \in [B] \right\}.$$

PROTOCOL 4: A linear TP-PSD protocol for resource constrained P_1

In the above protocol, we can make the decision between instantiating F using a PRP or a PRF based on practical considerations. For the same domain and codomain $\mathbb{F}$, PRFs generally offer better performance, making them the preferred choice. However, using a PRF requires the assumption that the size q of the field $\mathbb{F}$ is large enough such that the probability that distinct elements in the same bin map to distinct elements under F except with negligible probability.

Recall that, in practice, we may choose the maximum number of actual items in each bin to be $m = 27$ for $n \leq 2^{26}$. Thus, assuming F is a uniformly random function, the probability that there are two distinct elements in the same bin mapping to the same element under F is at most

$$b\left(1 - \prod_{k=0}^{27}\left(1 - \frac{k}{q}\right)\right) < b\left(\frac{\sum_{k=0}^{27} k}{q}\right) = \frac{378b}{q} = \frac{480.06n}{q}.$$

To make this probability $< 2^{-40}$, it suffices to choose $q \geq (2^{40})(480.06)n$. This means that if the field $\mathbb{F}$ already satisfies $|\mathbb{F}| \geq (2^{40})(480.06)n$, then using a PRF is likely the optimal choice. However, if it is instead necessary to enlarge the field $\mathbb{F}$ to satisfy this condition, using a PRP may be more efficient, depending on the performance trade-offs.

5.3. **Correctness and security.**

Proposition 7. *Protocol 4 is correct except with negligible probability.*

Proof. By the choices of b and m, hashing succeeds except with negligible probability. Similar to the proof of Proposition 3, we shall consider two cases depending on whether a bin B_i contains an actual element x_i or the dummy element d.

First, assume a bin B_i contains an actual item x_i. If $x_i \in S_2$, then x_i must be in bin B'_i, hence $(X - F_{k_2}(x_i))$ is a factor of the polynomial $p_i(X)$ and $p_i(e_i) = p_i(F_{k_2}(x_i)) = 0$. Let $j = \pi_{k_1}^{-1}(i)$. Then, in step 8, Q and P_2 obtain shares γ'_j and δ'_j such that

$$\gamma'_j + \delta'_j = p'_j(e'_j) = p_i(e_i) = 0,$$

and so Q has no output for this j.

On the other hand, if $x_i \notin S_2$, then by our asssumption on $|\mathbb{F}|$, $(X - F_{k_2}(x_i))$ is not a factor of $p_i(X)$ except with negligible probability, hence $p_i(e_i) \neq 0$. Let $j = \pi_{k_1}^{-1}(i)$ as above. We still have $\gamma'_j + \delta'_j = p'_j(e'_j)$. Thus, in step 9, Q and P_2 obtain shares ε'_j and ζ'_j such that

$$\varepsilon'_j + \zeta'_j = (\alpha'_j + \beta'_j)(\gamma'_j + \delta'_j) = x_i p'_j(e'_j),$$

and Q obtains the output

$$\frac{\varepsilon'_j + \zeta'_j}{\gamma'_j + \delta'_j} = \frac{x_i p'_j(e'_j)}{p'_j(e'_j)} = x_i.$$

Next, assume we are in the case where a bin B_i contains the dummy element d. Since $(X - F_{k_2}(d))$ is a factor of $p_i(X)$, we see that, for $j = \pi_{k_1}^{-1}(i)$,

$$\gamma'_j + \delta'_j = p'_j(e'_j) = p_i(e_i) = p_i(F_{k_2}(d)) = 0,$$

and Q has no output for this j. □

Proposition 8. *Protocol 4 is secure in the* $(\mathcal{F}_{\mathsf{ssMult}}, \mathcal{F}_{\mathsf{ssOPE}})$*-hybrid model against a semi-honest adversary corrupting a single party.*

Proof. Since P_1 receives no messages during the execution of Protocol 4, the protocol is trivially secure against a corrupted P_1.

The messages received by P_2 are k_1, k_2, β'_i, δ'_i and ζ'_i. A simulator $\mathcal{S}$ for a corrupted P_2 simply simulates k_1 and k_2 using random keys for π and F respectively, and simulates β'_i, δ'_i and ζ'_i using independent and uniformly random elements of $\mathbb{F}$.

Now, consider a corrupted Q. Q receives the messages α'_i, e'_i, γ'_i, δ'_i, ε'_i and ζ'_i. The elements α'_i are clearly uniformly random elements of $\mathbb{F}$ from the view of Q. Since F is either a secure PRP or a secure PRF,

$$e'_i = e_{\pi_{k_1}(i)} = F_{k_2}(x_{\pi_{k_1}(i)})$$

for $i \in [b]$ are indistinguishable from b uniformly random elements of $\mathbb{F}$, which are distinct when F is a PRP. Fix some i, and let $j = \pi_{k_1}(i)$. Note that (γ'_i, δ'_i) is uniformly random subject to the constraint

$$\gamma'_i + \delta'_i = p'_i(e'_i) = p_j(e_j).$$

As $p_j(X)$ is multiplied by the uniformly random element $r_j \in \mathbb{F} \setminus \{0\}$ which is unknown to Q, $p_j(e_j)$ is either 0 or a uniformly random element of $\mathbb{F} \setminus \{0\}$ in Q's view. From the proof of Proposition 7, we are in the latter case when $x_j \in S_1 \setminus S_2$ and in the former case otherwise. Next, ε'_i and ζ'_i are uniformly random under the constraint

$$\varepsilon'_i + \zeta'_i = x_j p_j(e_j).$$

We can thus construct a simulator S for a corrupted Q as follows. The elements α'_i and e'_i are simulated using independent and uniformly random elements of $\mathbb{F}$, with the additional condition that the elements e'_i are distinct when F is a PRP. Next, for each i, $\mathcal{S}$ picks independent and uniformly random elements $\gamma'_i, \varepsilon'_i \xleftarrow{\$} \mathbb{F}$. Let $\{z_1, \ldots, z_d\}$ be Q's output. $\mathcal{S}$ then picks a random injective map $\rho : [d] \to [b]$, and defines

$$\begin{aligned} &\delta'_i \xleftarrow{\$} \mathbb{F} \setminus \{-\gamma'_i\} && \text{if } i = \rho(j) \text{ for some } j \in [d], \\ &\delta'_i = -\gamma'_i && \text{otherwise,} \end{aligned}$$

and

$$\zeta'_i = \begin{cases} z_j(\gamma'_i + \delta'_i) - \varepsilon'_i & \text{if } i = \rho(j) \text{ for some } j \in [d], \\ -\varepsilon'_i & \text{otherwise.} \end{cases}$$

□

6. A Linear Third Party Private Symmetric Difference Protocol

6.1. **An overview.** In this section, we will introduce a third-party private symmetric difference (TP-PSymD) protocol by building upon the ideas introduced in Section 4. In a TP-PSymD protocol, an inputless third party Q should receive the output $S_1 \triangle S_2$ that is the symmetric difference of two private sets S_1 and S_2 held by P_1 and P_2 respectively.

Recall that, in Protocol 3, P_1 and P_2 use $\mathcal{F}_{\mathsf{ssOPE}}$ and $\mathcal{F}_{\mathsf{ssMult}}$ to generate shares γ_i, δ_i, ε_i and ζ_i for $1 \leq i \leq B$ which collectively encode the set $S_1 \setminus S_2$. These shares are never reconstructed by P_1 and P_2, so they do not learn $S_1 \setminus S_2$. The idea to achieve TP-PSymD is for P_1 and P_2 to generate additional shares γ_i, δ_i, ε_i and ζ_i for $B+1 \leq i \leq 2B$ which encode the set $S_2 \setminus S_1$. This can be realized using the same techniques employed in Protocol 3.

By sending these shares to the third party Q, he can then compute the symmetric difference $S_1 \triangle S_2$ as the union of $S_1 \setminus S_2$ and $S_2 \setminus S_1$. However, as explained in Section 4, this may leak information about the sets S_1 and S_2 due to the properties of cuckoo hashing. To prevent this leakage, P_1 and P_2 shuffle the shares using a PRP with a common key before sending them to Q.

It is essential that all $2B$ tuples of shares, those encoding both $S_1 \setminus S_2$ and $S_2 \setminus S_1$, are shuffled together. This ensures that Q cannot distinguish which elements belong

to $S_1 \setminus S_2$ and which belong to $S_2 \setminus S_1$, thereby satisfying the privacy guarantees required by a TP-PSymD protocol.

6.2. **Protocol details.** We use a similar setup as Section 3.2, except that we replace π with a PRP with domain and codomain $[2b]$, i.e. $\pi : \mathcal{K}_1 \times [2b] \to [2b]$. For $m_1 \leq m_2$, we will use the notation $[m_1, m_2]$ to denote the set $\{m_1, m_1+1, \ldots, m_2\}$. Additionally, we choose 3 independent random hash functions $h_4, h_5, h_6 : \mathbb{F} \to [b+1, 2b]$.

We now present our TP-PSymD protocol:

(1) P_1 uses cuckoo hashing with the hash functions h_1, h_2 and h_3 to hash the items of S_1 into b bins $B_1, \ldots, B_b$. Empty bins are filled with the dummy element d.
(2) P_2 uses simple hashing with the hash functions h_1, h_2 and h_3 to hash the items of S_2 into $B'_1, \ldots, B'_b$.
(3) For each $i \in [b]$, P_1 generates additive shares α_i and β_i for the unique item x_i in bin B_i and sends β_i to P_2.
(4) For each $i \in [b]$, P_2 picks a random $r_i \xleftarrow{\$} \mathbb{F} \setminus \{0\}$ and computes the polynomial
$$p_i(X) = r_i(X - d) \prod_{y_i \in B'_i} (X - y_i).$$
(5) For each $i \in [b]$, P_1 and P_2 invoke $\mathcal{F}_{\mathsf{ssOPE}}$ with
   - P_1 having input x_i,
   - P_2 having input $p_i(X)$.

   Let γ_i and δ_i be the outputs obtained by P_1 and P_2 respectively from $\mathcal{F}_{\mathsf{ssOPE}}$.
(6) For each $i \in [b]$, P_1 and P_2 invoke $\mathcal{F}_{\mathsf{ssMult}}$ with
   - P_1 having inputs α_i and γ_i,
   - P_2 having inputs β_i and δ_i,

   to obtain shares ε_i and ζ_i for P_1 and P_2 respectively.
(7) P_2 uses cuckoo hashing with the hash functions h_4, h_5 and h_6 to hash the items of S_2 into b bins $B_{b+1}, \ldots, B_{2b}$. Empty bins are filled with the dummy element d.
(8) P_1 uses simple hashing with the hash functions h_4, h_5 and h_6 to hash the items of S_1 into $B'_{b+1}, \ldots, B'_{2b}$.
(9) For each $i \in [b+1, 2b]$, P_2 generates additive shares α_i and β_i for the unique item y_i in bin B_i and sends α_i to P_1.
(10) For each $i \in [b+1, 2b]$, P_1 picks a random $r_i \xleftarrow{\$} \mathbb{F} \setminus \{0\}$ and computes the polynomial
$$p_i(X) = r_i(X - d) \prod_{x_i \in B'_i} (X - x_i).$$
(11) For each $i \in [b+1, 2b]$, P_1 and P_2 invoke $\mathcal{F}_{\mathsf{ssOPE}}$ with
   - P_1 having input $p_i(X)$,
   - P_2 having input y_i.

   Let γ_i and δ_i be the outputs obtained by P_1 and P_2 respectively from $\mathcal{F}_{\mathsf{ssOPE}}$.
(12) For each $i \in [b+1, 2b]$, P_1 and P_2 invoke $\mathcal{F}_{\mathsf{ssMult}}$ with
   - P_1 having inputs α_i and γ_i,

- P_2 having inputs β_i and δ_i,

to obtain shares ε_i and ζ_i for P_1 and P_2 respectively.

(13) P_1 picks a random key $k \in \mathcal{K}_1$ and sends k to P_2.

(14) P_1 and P_2 shuffle γ_i, ε_i and δ_i, ζ_i respectively using the permutation $\pi_k : [2b] \to [2b]$, i.e. they compute

$$\gamma'_i = \gamma_{\pi_k(i)}, \quad \varepsilon'_i = \varepsilon_{\pi_k(i)}, \quad \delta'_i = \delta_{\pi_k(i)}, \quad \zeta'_i = \zeta_{\pi_k(i)}$$

and send the shuffled elements to Q.

(15) Q outputs

$$\left\{ \frac{\varepsilon'_i + \zeta'_i}{\gamma'_i + \delta'_i} \text{ such that } \gamma'_i + \delta'_i \neq 0 : \ i \in [2b] \right\}.$$

PROTOCOL 5: A fast linear-time TP-PSymD protocol

Just like in Protocol 3, here, we can instantiate $\mathcal{F}_{\text{ssOPE}}$ and $\mathcal{F}_{\text{ssMult}}$ with protocols that are assisted by Q for the best efficiency, or with unasssisted protocols to obtain a TP-PSymD protocol that is suitable for resource-constrained Q.

6.3. **Correctness and security.**

Proposition 9. *Protocol 5 is correct except with negligible probability.*

Proof. By the choices of b and m, hashing succeeds except with negligible probability. Analogous to the proof of Proposition 5, we observe that, for $i \in [b]$,

$$\gamma_i + \delta_i = \begin{cases} p_i(x_i) & \text{if } B_i \text{ contains } x_i \neq d \text{ and } x_i \notin S_2, \\ 0 & \text{otherwise,} \end{cases}$$

and

$$\varepsilon_i + \zeta_i = \begin{cases} x_i p_i(x_i) & \text{if } B_i \text{ contains } x_i \neq d \text{ and } x_i \notin S_2, \\ 0 & \text{otherwise.} \end{cases}$$

Thus, for $j = \pi_k^{-1}(i)$ with $i \in [b]$, if B_i contains $x_i \in S_1 \setminus S_2$, Q obtains the output $(\varepsilon'_j + \zeta'_j)/(\gamma'_j + \delta'_j) = x_i$, and has no output otherwise.

Similarly, for $i \in [b+1, 2b]$,

$$\gamma_i + \delta_i = \begin{cases} p_i(y_i) & \text{if } B_i \text{ contains } y_i \neq d \text{ and } y_i \notin S_1, \\ 0 & \text{otherwise,} \end{cases}$$

and

$$\varepsilon_i + \zeta_i = \begin{cases} y_i p_i(y_i) & \text{if } B_i \text{ contains } y_i \neq d \text{ and } y_i \notin S_1, \\ 0 & \text{otherwise.} \end{cases}$$

It follows that, for $j = \pi_k^{-1}(i)$ with $i \in [b+1, 2b]$, if B_i contains $y_i \in S_2 \setminus S_1$, Q obtains the output $(\varepsilon'_j + \zeta'_j)/(\gamma'_j + \delta'_j) = y_i$, and has no output otherwise.

Therefore, Q's output in step 15 of the protocol is $(S_1 \setminus S_2) \cup (S_2 \setminus S_1) = S_1 \triangle S_2$, as required. □

Proposition 10. *Protocol 5 is secure in the $(\mathcal{F}_{\mathsf{ssMult}}, \mathcal{F}_{\mathsf{ssOPE}})$-hybrid model against a semi-honest adversary corrupting a single party.*

Proof. In Protocol 5, the only messages received by P_1 are γ_i and ε_i for $i \in [b]$ in steps 5 and 6, α_i for $i \in [b+1, 2b]$ in step 9, and γ_i and ε_i for $i \in [b+1, 2b]$ in steps 11 and 12. In P_1's view, all these values are independent and uniformly distributed over $\mathbb{F}$. Therefore, a simulator $\mathcal{S}$ for a corrupted P_1 can perfectly simulate the

protocol by generating independent and uniformly random elements of $\mathbb{F}$ for these messages when needed.

Similarly, the only messages received by P_2 are β_i for $i \in [b]$ in step 3, δ_i and ζ_i for $i \in [b]$ in steps 5 and 6, δ_i and ζ_i for $i \in [b+1, 2b]$ in steps 11 and 12, and k in step 13. Except for k, all these values are independent and uniformly distributed over $\mathbb{F}$, while k is a random key in $\mathcal{K}_1$. Thus, for a corrupted P_2, a simulator $\mathcal{S}$ can simply generate independent and uniformly random elements of $\mathbb{F}$ for the first set of messages and sample k from $\mathcal{K}_1$.

Now, consider a corrupted Q. Q receives the messages γ'_i, ε'_i from P_1 and δ'_i, ζ'_i from P_2. Fix some i, and let $j = \pi_k(i)$. Analogous to the proof of Proposition 6, we observe that the pair (γ'_i, δ'_i) is uniformly random subject to the constraint that

$$\gamma'_i + \delta'_i = \gamma_j + \delta_j = p_j(w_j)$$

where

$$w_j = \begin{cases} x_j & \text{if } j \in [b], \\ y_j & \text{if } j \in [b+1, 2b]. \end{cases}$$

From the proof of Proposition 9, $p_j(w_j) \neq 0$ if and only if $w_j \in S_1 \triangle S_2$. In this case, $p_j(w_j) \in \mathbb{F} \setminus \{0\}$ is uniformly random to Q since $r_j \in \mathbb{F} \setminus \{0\}$ is uniformly random. Similarly, $(\varepsilon'_i, \zeta'_i)$ is uniformly random subject to the constraint

$$\varepsilon'_i + \zeta'_i = \varepsilon_j + \zeta_j = w_j p_j(w_j).$$

We thus follow the approach in the proof of Proposition 6 to construct a simulator $\mathcal{S}$ for a corrupted Q. $\mathcal{S}$ picks independent and uniformly random elements $\gamma'_i, \varepsilon'_i \xleftarrow{\$} \mathbb{F}$ for each $i \in [2b]$. Suppose $\{z_1, \ldots, z_d\}$ is Q's output from the ideal functionality. $\mathcal{S}$ then picks a random injective map $\rho : [d] \to [2b]$, and computes

$$\begin{aligned} &\delta'_i \xleftarrow{\$} \mathbb{F} \setminus \{-\gamma'_i\} && \text{if } i = \rho(j) \text{ for some } j \in [d], \\ &\delta'_i = -\gamma'_i && \text{otherwise,} \end{aligned}$$

and

$$\zeta'_i = \begin{cases} z_j(\gamma'_i + \delta'_i) - \varepsilon'_i & \text{if } i = \rho(j) \text{ for some } j \in [d], \\ -\varepsilon'_i & \text{otherwise.} \end{cases}$$

□

7. Performance Evaluation

To evaluate the performance of our protocols, we implemented Protocols 2 to 5 in C++ using the NTL [45], libOTe [42] and OpenSSL libraries.

In our implementations, we instantiate $\mathcal{F}_{\mathsf{ssOPE}}$ and $\mathcal{F}_{\mathsf{ssMult}}$ in Protocols 3 and 5 using both assisted and unassisted versions. We refer to the protocols instantiated with the assisted versions as Protocols 3a and 5a, and those with the unassisted versions as Protocols 3b and 5b.

Protocol 1 is used as the assisted ssOPE protocol and a variant of an OPE protocol by Chang and Lu [6] as the unassisted ssOPE protocol. To achieve the ssMult functionality, we generate Beaver triples using oblivious linear evaluation and use them to perform multiplication on secret shared values. The oblivious transfers (OTs) required by the unassisted ssOPE protocol are performed using the OT extension protocol by Ishai et al. [22], allowing a large number of OTs to be performed efficiently with a small number of base OTs.

We use a PRP F in our implementation of Protocol 4, since the field size required to achieve an error probability of $< 2^{-40}$ when using a PRF exceeds the limits of

hardware-supported arithmetic and requires multi-precision computation, which significantly degrades performance.

We perform our experiments on a machine with two processors, each with 12 cores operating at a base frequency of 2.20 GHz. To evaluate both single-threaded and multi-threaded performance, we benchmark our protocols using configurations with 1 thread and 8 threads.

To demonstrate the usefulness of Protocols 3b and 5b for a resource-constrained Q, we use cpulimit to cap Q's CPU usage at 100% of a single core, and tc to simulate a network bandwidth of 100Mbps for both upload and download. When benchmarking Protocol 4, we apply similar constraints to P_1 instead of Q.

For each evaluated protocol–configuration combination, we perform 5 runs and report the mean running time. The results of our experiments are presented in Table 1.

We compare the running times and communication costs of our protocols to state-of-the-art protocols achieving the PSU, TP-PSD and TP-PSymD functionalities and the results are shown in Table 2. The results presented for the PSU protocols proposed by Zhang et al. [49] and Jia et al. [23] are obtained from their respective papers. Note that the results for Zhang et al. [49] assume that Beaver triples are pre-generated, and their generation is excluded from the reported running times and communication costs. Compared to state-of-the-art two-party PSU protocols, Protocol 2 demonstrates a 2.5×–3.0× improvement with a single thread and a 4.5×–5.1× improvement with 8 threads on the evaluated set sizes.

We use a single thread for the TP-PSD comparison and 8 threads for the TP-PSymD comparison, following prior work. The comparisons are limited to small set sizes, as prior approaches do not scale well to larger inputs. Protocol 3a and Protocol 5a achieve a greater than 80× speedup over the current state-of-the-art TP-PSD and TP-PSymD protocols, respectively, for the evaluated set sizes. This can be partly attributed to the heavy use of public key operations in prior approaches. As previous protocols are non-linear, our linear-time protocols show even greater performance gains as set sizes grow.

		Protocol	Input size n			
			2^{18}	2^{20}	2^{22}	2^{24}
Running time (s)	single thread	Prot. 2	4.32	16.89	69.45	281.48
	8 threads	Prot. 2	1.09	4.48	20.06	81.46
Communication cost (GB)		Prot. 2	0.35	1.40	5.60	23.02

(A) Server-aided PSU performance

		Protocol	Input size n			
			2^{18}	2^{20}	2^{22}	2^{24}
Running time (s)	single thread	Prot. 3a	4.10	17.04	69.24	276.22
	8 threads	Prot. 3a	1.08	4.18	19.63	79.25
	resource constrained Q	Prot. 3a	14.62	58.94	235.43	975.28
		Prot. 3b	8.17	28.55	129.87	588.61
	resource constrained P_1	Prot. 3a	22.89	90.32	360.93	1454.57
		Prot. 4	9.86	33.42	133.68	592.00
Communication cost (GB)		Prot. 3a	0.34	1.35	5.40	22.23
		Prot. 3b	14.70	58.81	235.23	973.31
		Prot. 4	14.70	58.81	235.23	973.31

(B) TP-PSD performance

		Protocol	Input size n			
			2^{18}	2^{20}	2^{22}	2^{24}
Running time (s)	single thread	Prot. 5a	7.48	29.62	124.36	507.78
	8 threads	Prot. 5a	1.84	7.67	36.22	147.69
	resource constrained Q	Prot. 5a	30.36	119.23	474.60	1936.18
		Prot. 5b	18.05	63.81	265.06	1050.48
Communication cost (GB)		Prot. 5a	0.67	2.70	10.80	44.45
		Prot. 5b	29.40	117.62	470.47	1946.62

(C) TP-PSymD performance

TABLE 1. Running times and communication costs for server-aided PSU, TP-PSD and TP-PSymD

		Protocol	Input size n		
			2^{18}	2^{20}	2^{22}
Running time (s)	single thread	Zhang et al. [49]	10.82	44.78	—
		Jia et al. [23]	12.98	49.38	202.49
		Protocol 2	4.32	16.89	69.45
	8 threads	Zhang et al. [49]	4.90	22.81	—
		Protocol 2	1.09	4.48	20.06
Communication cost (MB)		Zhang et al. [49]	103.9	414.4	—
		Jia et al. [23]	577.7	2430.5	10204.7
		Protocol 2	358.1	1432.6	5730.2

(A) Comparison of PSU protocols

		Protocol	Input size n		
			2^7	2^8	2^9
Running time (s)	single thread	Yeo & Ying [47]	6.298	6.307	7.313
		Protocol 3a	0.051	0.053	0.057
Communication cost (MB)		Yeo & Ying [47]	1.01	1.02	1.02
		Protocol 3a	0.17	0.34	0.68

(B) Comparison of TP-PSD protocols

		Protocol	Input size n		
			2^7	2^8	2^9
Running time (s)	8 threads	Yeo & Ying [47]	12.103	18.269	34.949
		Protocol 5a	0.148	0.150	0.154
Communication cost (MB)		Yeo & Ying [47]	2.21	3.46	5.94
		Protocol 5a	0.34	0.68	1.35

(C) Comparison of TP-PSymD protocols

TABLE 2. Comparison of running times and communication costs with prior work

References

[1] Josh Alman, Ran Duan, Virginia Vassilevska Williams, Yinzhan Xu, Zixuan Xu, and Renfei Zhou. More asymmetry yields faster matrix multiplication. pages 2005–2039, 2025.

[2] Diego F. Aranha, Chuanwei Lin, Claudio Orlandi, and Mark Simkin. Laconic private set-intersection from pairings. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, pages 111–124. ACM, 2022.

[3] Donald Beaver. Efficient multiparty protocols using circuit randomization. In *Advances in Cryptology — CRYPTO '91*, pages 420–432, Berlin, Heidelberg, 1992. Springer Berlin Heidelberg.

[4] Ran Canetti. Universally composable security: a new paradigm for cryptographic protocols. In *Proceedings 42nd IEEE Symposium on Foundations of Computer Science*, pages 136–145, 2001.

[5] Ran Canetti, Asaf Cohen, and Yehuda Lindell. A simpler variant of universally composable security for standard multiparty computation. In *Advances in Cryptology — CRYPTO 2015*, pages 3–22. Springer Berlin Heidelberg, 2015.

[6] Yan-Cheng Chang and Chi-Jen Lu. Oblivious polynomial evaluation and oblivious neural learning. *Theoretical Computer Science*, 341(1):39–54, 2005.

[7] Melissa Chase and Peihan Miao. Private set intersection in the internet setting from lightweight oblivious PRF. In *Advances in Cryptology — CRYPTO 2020*, pages 34–63. Springer International Publishing, 2020.

[8] Hao Chen, Kim Laine, and Peter Rindal. Fast private set intersection from homomorphic encryption. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, CCS '17, pages 1243–1255. Association for Computing Machinery, 2017.

[9] Kai Chen, Yongqiang Li, and Mingsheng Wang. An efficient toolkit for computing third-party private set intersection. In *International Conference on Cryptology in India (Indocrypt '24)*, pages 258–281. Springer Nature Switzerland, 2024.

[10] Wutichai Chongchitmate, Yuval Ishai, Steve Lu, and Rafail Ostrovsky. PSI from ring-OLE. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, pages 531–545. ACM, 2022.

[11] Ivan Damgård, Valerio Pastro, Nigel Smart, and Sarah Zakarias. Multiparty computation from somewhat homomorphic encryption. In Reihaneh Safavi-Naini and Ran Canetti, editors, *Advances in Cryptology — CRYPTO 2012*, pages 643–662. Springer Berlin Heidelberg, 2012.

[12] Alex Davidson and Carlos Cid. An efficient toolkit for computing private set operations. In *Australasian Conference on Information Security and Privacy*, pages 261–278. Springer International Publishing, 2017.

[13] Changyu Dong, Liqun Chen, and Zikai Wen. When private set intersection meets big data: an efficient and scalable protocol. In *Proceedings of the 2013 ACM SIGSAC conference on Computer & communications security*, pages 789–800. ACM, 2013.

[14] Dimitris Fotakis, Rasmus Pagh, Peter Sanders, and Paul Spirakis. Space efficient hash tables with worst case constant access time. In *Annual Symposium on Theoretical Aspects of Computer Science*, pages 271–282. Springer Berlin Heidelberg, 2003.

[15] Dimitris Fotakis, Rasmus Pagh, Peter Sanders, and Paul Spirakis. Space efficient hash tables with worst case constant access time. *Theory of Computing Systems*, 38(2):229–248, 2005.

[16] Michael J. Freedman, Kobbi Nissim, and Benny Pinkas. Efficient private matching and set intersection. In *Advances in Cryptology — EUROCRYPT 2004*, pages 1–19. Springer, Berlin, Heidelberg, 2004.

[17] Keith Frikken. Privacy-preserving set union. In *International Conference on Applied Cryptography and Network Security*, pages 237–252. Springer Berlin Heidelberg, 2007.

[18] Gayathri Garimella, Payman Mohassel, Mike Rosulek, Saeed Sadeghian, and Jaspal Singh. Private set operations from oblivious switching. In *IACR International Conference on Public-Key Cryptography*, pages 591–617. Springer International Publishing, 2021.

[19] Gayathri Garimella, Benny Pinkas, Mike Rosulek, Ni Trieu, and Avishay Yanai. Oblivious key-value stores and amplification for private set intersection. In *Advances in Cryptology — CRYPTO 2021: 41st Annual International Cryptology Conference, CRYPTO 2021*, pages 395–425. Springer International Publishing, 2021.

[20] Niv Gilboa. Two party RSA key generation. In *Advances in Cryptology — CRYPTO '99*, pages 116–129, Berlin, Heidelberg, 1999. Springer Berlin Heidelberg.

[21] Kyle Hogan, Noah Luther, Nabil Schear, Emily Shen, David Stott, Sophia Yakoubov, and Arkady Yerukhimovich. Secure multiparty computation for cooperative cyber risk assessment. In *2016 IEEE Cybersecurity Development (SecDev)*, pages 75–76. IEEE, 2016.

[22] Yuval Ishai, Joe Kilian, Kobbi Nissim, and Erez Petrank. Extending oblivious transfers efficiently. In *Annual International Cryptology Conference*, pages 145–161. Springer Berlin Heidelberg, 2003.

[23] Yanxue Jia, Shi-Feng Sun, Hong-Sheng Zhou, and Dawu Gu. Scalable private set union, with stronger security. In *33rd USENIX Security Symposium (USENIX Security '24)*, pages 6471–6488. USENIX Association, 2024.

[24] Lea Kissner and Dawn Song. Privacy-preserving set operations. In *Advances in Cryptology — CRYPTO 2005*, pages 241–257. Springer Berlin Heidelberg, 2005.

[25] Vladimir Kolesnikov, Ranjit Kumaresan, Mike Rosulek, and Ni Trieu. Efficient batched oblivious PRF with applications to private set intersection. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS'16)*, pages 818–829. ACM, 2016.

[26] Vladimir Kolesnikov, Mike Rosulek, Ni Trieu, and Xiao Wang. Scalable private set union from symmetric-key techniques. In *International conference on the theory and application of cryptology and information security*, pages 636–666. Springer International Publishing, 2019.

[27] Arjen Lenstra and Tim Voss. Information security risk assessment, aggregation, and mitigation. In *Australasian Conference on Information Security and Privacy*, pages 391–401. Springer Berlin Heidelberg, 2004.

[28] Han Liang, Zhenhua Liu, Baocang Wang, Wenxin Wang, Yujing Yuan, and Tao Liang. Practical third-party private set intersection protocol from homomorphic encryption. In *International Conference on Frontiers in Cyber Security*, pages 44–61. Springer Nature Singapore, 2024.

[29] Rasoul Akhavan Mahdavi, Nils Lukas, Faezeh Ebrahimianghazani, Thomas Humphries, Bailey Kacsmar, John Premkumar, Xinda Li, Simon Oya, Ehsan Amjadian, and Florian Kerschbaum. PEPSI: Practically efficient private set intersection in the unbalanced setting. In *33rd USENIX Security Symposium (USENIX Security '24)*, pages 6453–6470. USENIX Association, 2024.

[30] Payman Mohassel and Saeed Sadeghian. How to hide circuits in mpc an efficient framework for private function evaluation. In *Advances in Cryptology — EUROCRYPT 2013*, pages 557–574, Berlin, Heidelberg, 2013. Springer Berlin Heidelberg.

[31] Moni Naor and Benny Pinkas. Oblivious transfer and polynomial evaluation. In *Proceedings of the thirty-first annual ACM symposium on Theory of computing*, pages 245–254. ACM, 1999.

[32] Moni Naor and Benny Pinkas. Oblivious polynomial evaluation. *SIAM Journal on Computing*, 35(5):1254–1281, 2006.

[33] Rasmus Pagh and Flemming Friche Rodler. Cuckoo hashing. In *Algorithms — ESA 2001*, pages 121–133, Berlin, Heidelberg, 2001. Springer Berlin Heidelberg.

[34] Rasmus Pagh and Flemming Friche Rodler. Cuckoo hashing. *Journal of Algorithms*, 51(2):122–144, 2004.

[35] Benny Pinkas, Mike Rosulek, Ni Trieu, and Avishay Yanai. SpOT-light: lightweight private set intersection from sparse OT extension. In *Advances in Cryptology — CRYPTO 2019: 39th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18–22, 2019, Proceedings, Part III 39*, pages 401–431. Springer International Publishing, 2019.

[36] Benny Pinkas, Thomas Schneider, Gil Segev, and Michael Zohner. Phasing: Private set intersection using permutation-based hashing. In *24th USENIX Security Symposium (USENIX Security '15)*, pages 515–530, 2015.

[37] Benny Pinkas, Thomas Schneider, Oleksandr Tkachenko, and Avishay Yanai. Efficient circuit-based PSI with linear communication. In *Advances in Cryptology — EUROCRYPT 2019: 38th Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 122–153. Springer International Publishing, 2019.

[38] Benny Pinkas, Thomas Schneider, Christian Weinert, and Udi Wieder. Efficient circuit-based PSI via cuckoo hashing. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 125–157. Springer International Publishing, 2018.

[39] Benny Pinkas, Thomas Schneider, and Michael Zohner. Faster private set intersection based on OT extension. In *23rd USENIX Security Symposium (USENIX Security '14)*, pages 797–812, 2014.

[40] Benny Pinkas, Thomas Schneider, and Michael Zohner. Scalable private set intersection based on OT extension. *ACM Transactions on Privacy and Security (TOPS)*, 21(2):1–35, 2018.
[41] Srinivasan Raghuraman and Peter Rindal. Blazing fast PSI from improved OKVS and subfield VOLE. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, pages 2505–2517. ACM, 2022.
[42] Peter Rindal and Lance Roy. libOTe: an efficient, portable, and easy to use oblivious transfer library, 2026. `https://github.com/osu-crypto/libOTe`.
[43] Peter Rindal and Phillipp Schoppmann. VOLE-PSI: Fast OPRF and circuit-PSI from vector-OLE. In *Advances in Cryptology — EUROCRYPT 2021*, pages 901–930. Springer International Publishing, 2021.
[44] Adi Shamir. How to share a secret. *Communications of the ACM*, 22(11):612–613, 1979.
[45] Victor Shoup. NTL: A library for doing number theory, 2025. `https://libntl.org/`.
[46] Foo Yee Yeo and Jason H. M. Ying. Third-party private set intersection. In *2023 IEEE International Symposium on Information Theory (ISIT)*, pages 1633–1638. IEEE, 2023.
[47] Foo Yee Yeo and Jason H. M. Ying. Private set intersection and other set operations in the third party setting. In *34th USENIX Security Symposium (USENIX Security '25)*, pages 7723–7742. USENIX Association, 2025.
[48] Foo Yee Yeo and Jason H. M. Ying. A round-optimal near-linear third-party private set intersection protocol. In *Provable and Practical Security*, pages 279–298. Springer-Verlag, 2025.
[49] Cong Zhang, Yu Chen, Weiran Liu, Min Zhang, and Dongdai Lin. Linear private set union from multi-query reverse private membership test. In *32nd USENIX Security Symposium (USENIX Security '23)*, pages 337–354. USENIX Association, 2023.
[50] Yongjun Zhao and Sherman S. M. Chow. Are you the one to share? Secret transfer with access structure. *Proceedings on Privacy Enhancing Technologies*, 2017(1):149–169, 2017.
[51] Yongjun Zhao and Sherman S. M. Chow. Can you find the one for me? In *Proceedings of the 2018 Workshop on Privacy in the Electronic Society*, pages 54–65, 2018.

Foo Yee Yeo, Seagate Technology, Singapore
Email address: `fooyee.yeo@seagate.com`

Jason H. M. Ying, Seagate Technology, Singapore
Email address: `jasonhweiming.ying@seagate.com`